

SMT粗分机接口调用说明书

文档版本: v2.0

最后更新: 2026-03-21

适用系统: SMT粗分机系统

设备ID范围: PIPELINE01/ARM01/ARM02 (左侧), PIPELINE02/ARM03/ARM04 (右侧)

目录

- 1. 接口概览
- 2. 数据模型定义
- 3. 命令接口
- 4. 查询接口
- 5. 回调接口
- 6. 事件推送接口
- 7. 错误处理
- 8. 完整示例代码
- 9. 附录

1. 接口概览

1.1 系统概述

SMT粗分机系统是SMT生产线中的自动化设备，主要负责电子物料的自动分拣、搬运和存储。系统包含两个独立的粗分机单元，每个单元配备一个流水线和两个机械臂。

系统组成:

左侧半边 (单串杆和四串杆) :

- PIPELINE01 (流水线1) : 负责物料在进料位置和出料位置之间的传输
- ARM01 (进料机械臂1) : 负责从串杆位置抓取物料，进行扫码、尺寸检测、测厚、NG判定、放置到流水线进料位置
- ARM02 (出料机械臂1) : 负责从流水线出料位置抓取物料，放置到单层货架料箱

右侧半边 (单串杆) :

- PIPELINE02 (流水线2) : 负责物料在进料位置和出料位置之间的传输
- ARM03 (进料机械臂2) : 负责从串杆位置抓取物料，进行扫码、尺寸检测、测厚、NG判定、放置到流水线进料位置
- ARM04 (出料机械臂2) : 负责从流水线出料位置抓取物料，放置到单层货架料箱

主要作业类型:

- 进料NG流程 (扫码NG) : 串杆扫码NG → 放到NG缓存位
- 进料OK流程 (扫码OK) : 串杆扫码 → 放到流水线进料位置 → 尺寸检测和测厚
- 进料OK→移料流程 (尺寸检测OK) : 串杆扫码 → 放到流水线进料位置 → 尺寸检测和测厚OK → 流水线传输 → 放到料箱
- 进料NG流程 (尺寸检测/测厚NG) : 串杆扫码 → 放到流水线进料位置 → 尺寸检测/测厚NG → 从流水线进料位置移到NG缓存位
- 出料流程: 从流水线出料位置 → 放到料箱

1.2 接口分类

接口分类	接口名称	HTTP方法	说明
命令接口	发送搬运命令	POST /api/v1/command	向设备发送搬运、移动等任务命令
命令接口	取消命令	POST /api/v1/command/cancel	取消已发送但未执行的命令
查询接口	查询设备状态	GET /api/v1/status	查询流水线和机械臂的当前状态
回调接口	任务结果回传	POST [WES回调地址]	设备向WES回传任务执行结果
事件接口	事件推送	POST [WES事件上报地址]	设备向WES推送事件（如扫码完成、急停等）

1.3 通信协议

- 通信协议: HTTP/HTTPS
- 数据格式: JSON
- 字符编码: UTF-8
- Content-Type: application/json

2. 数据模型定义

2.1 PickAndPutCommand（搬运命令）

字段名	类型	必填	说明
device_id	string	是	设备ID（ARM01/ARM02/ARM03/ARM04）
command_id	string	是	命令ID，用于追踪和取消命令，建议格式：CMD-{YYYYMMDD}-{序号}
timestamp	string	是	时间戳（字符串格式，毫秒）
task_type	string	是	任务类型 (PICK_AND_PUT/SCAN/TEST/MOVE_FORWARD/MOVE_BACKWARD/MOVE_LEFT/MOVE_RIGHT)
priority	int	否	优先级（1-10，10最高），默认1
timeout	int	否	期望完成时间（毫秒），默认30000
source	object	是	源位置信息
target	object	是	目标位置信息

2.2 PickAndPutLocationInfo（位置信息）

字段名	类型	必填	说明
location_id	string	是	位置ID
location_type	string	是	位置类型 (INPUT_PLATFORM/NG_PLATFORM/PIPELINE_PLATFORM/OUTPUT_PLATFORM/BIN)
rack_id	string	否	料架编号，ECS追溯用（可选）
bin_id	string	否	箱体编号，ECS追溯用（可选）
bin_type	string	否	料箱类型（三格箱/六格箱）

字段名	类型	必填	说明
bin_cell_location	string	否	料箱格子位置（三格箱：1,2,7；六格箱：1,2,3,4,5,6）
reel_layer	string	否	目标料箱料盘层数
reel_thickness	string	否	当前料盘厚度（mm）
reel_diameter	string	否	当前料盘直径（如：15inch）
reel_totalthickness	string	否	当前料箱料盘总厚度

2.3 DeviceStatusItem（设备状态项）

字段名	类型	必填	说明
device_id	string	是	设备ID（PIPELINE01/ARM01/ARM02/PIPELINE02/ARM03/ARM04）
timestamp	long	是	时间戳（毫秒）
current_cmd_id	string	是	当前正在执行的命令ID
error_code	string	是	错误码（NONE表示无错误）
status	string	是	设备状态（IDLE/RUNNING/ERROR/OFFLINE）

2.4 TaskResultReport（任务结果报告）

字段名	类型	必填	说明
command_id	string	是	必须回传原指令ID
device_id	string	是	必须回传设备ID
result	string	是	执行结果（SUCCESS/FAILED）
finish_time	long	是	完成时间戳（毫秒）
message	string	否	业务回传信息
data	object	否	业务回传数据
error_detail	object	否	错误详情（当result=FAILED时必填）

2.5 TaskResultData（任务结果数据）

字段名	类型	必填	说明
actual_qty	int	是	实际搬运数量
location	string	否	实际搬运位置
barcode1 ~ barcode6	string	否	条码信息（最多6个条码）
reel_diameter	string	否	料盘尺寸（如：15inch）
reel_thickness	string	否	料盘厚度（mm）
pick_and_put_result	string	是	具体结果（PUT_FINISHED/PUT_FAILED/MOVE_FINISHED/MOVE_FAILED）

2.6 ErrorDetail（错误详情）

字段名	类型	必填	说明
error_code	string	是	错误码（0表示无错误，1001表示料盘尺寸检测异常，1002表示料盘厚度检测异常）
error_message	string	否	具体错误信息（如：扫码异常, 设备故障, 料盘尺寸检测异常, 料盘厚度检测异常等）

2.7 EventPush（事件推送）

字段名	类型	必填	说明
device_id	string	是	设备ID
event_type	string	是	事件类型（ESTOP_PRESSED/SCAN_COMPLETED）
timestamp	long	是	时间戳（毫秒）
data	object	否	业务负载数据

2.8 ApiResult（API响应）

字段名	类型	必填	说明
status	string	是	状态（SUCCESS/FAILED）
message	string	否	附加信息
code	string	是	响应码（200/400/503）
trace_id	string	否	供应商内部日志ID

3. 命令接口

3.1 发送搬运命令

接口地址: POST /api/v1/command

功能说明: 向设备发送搬运、移动等任务命令

3.1.1 进料NG命令（扫码NG：从串杆取出物料放到NG缓存位）

使用场景: 当扫码后发现物料不合格时，直接将物料移至NG缓存位

请求示例:

```
{
  "device_id": "ARM01",
  "command_id": "CMD-20251215-1001",
  "timestamp": "1702627300000",
  "task_type": "PICK_AND_PUT",
  "priority": 1,
  "timeout": 30000,
  "source": {
    "location_id": "STATION_INPUT1",
    "location_type": "INPUT_PLATFORM"
  },
  "target": {
    "location_id": "STATION_NG_PLATFORM1",
    "location_type": "NG_PLATFORM"
  }
}
```

字段说明:

- device_id: ARM01 (左侧进料机械臂) 或 ARM03 (右侧进料机械臂)
- task_type: PICK_AND_PUT (搬运任务)
- source.location_id: STATION_INPUT1 (串杆扫码位置)
- source.location_type: INPUT_PLATFORM (进料平台)
- target.location_id: STATION_NG_PLATFORM1 或 STATION_NG_PLATFORM2 (NG平台)
- target.location_type: NG_PLATFORM (NG平台)

响应示例:

```
{
  "status": "SUCCESS",
  "message": "Accepted",
  "code": "200",
  "trace_id": "DEV-LOG-1702627300000"
}
```

3.1.2 进料OK命令 (扫码OK: 从串杆取出物料放到流水线进料位置)

使用场景: 当扫码后物料合格时, 将物料移至流水线进料位置, 准备进行尺寸检测和测厚

请求示例:

```
{
  "device_id": "ARM01",
  "command_id": "CMD-20251215-1001",
  "timestamp": "1702627300000",
  "task_type": "PICK_AND_PUT",
  "priority": 1,
  "timeout": 30000,
  "source": {
    "location_id": "STATION_INPUT1",
    "location_type": "INPUT_PLATFORM"
  },
  "target": {
    "location_id": "STATION_PIPELINE1_INPUT1",
    "location_type": "PIPELINE_PLATFORM"
  }
}
```

字段说明:

- device_id: ARM01（左侧进料机械臂）或 ARM03（右侧进料机械臂）
- task_type: PICK_AND_PUT（搬运任务）
- source.location_id: STATION_INPUT1（串杆扫码位置）
- source.location_type: INPUT_PLATFORM（进料平台）
- target.location_id:
 - 左侧: STATION_PIPELINE1_INPUT1（流水线1进料位置1）或 STATION_PIPELINE1_INPUT2（流水线1进料位置2，四串杆使用）
 - 右侧: STATION_PIPELINE2_INPUT1（流水线2进料位置）
- target.location_type: PIPELINE_PLATFORM（流水线平台）

响应示例:

```
{
  "status": "SUCCESS",
  "message": "Accepted",
  "code": "200",
  "trace_id": "DEV-LOG-1702627300000"
}
```

3.1.3 进料NG命令（尺寸检测/测厚NG：从流水线进料位置取出物料放到NG缓存位）

使用场景: 当物料放置到流水线进料位置后，进行尺寸检测和测厚，如果检测不合格，需要将物料从流水线进料位置移到NG缓存位

请求示例:

```
{
  "device_id": "ARM01",
  "command_id": "CMD-20251215-1001",
  "timestamp": "1702627300000",
  "task_type": "PICK_AND_PUT",
  "priority": 1,
  "timeout": 30000,
  "source": {
    "location_id": "STATION_PIPELINE1_INPUT1",
    "location_type": "INPUT_PLATFORM"
  },
  "target": {
    "location_id": "STATION_NG_PLATFORM1",
    "location_type": "NG_PLATFORM"
  }
}
```

字段说明:

- device_id: ARM01 （左侧进料机械臂）或 ARM03 （右侧进料机械臂）
- task_type: PICK_AND_PUT （搬运任务）
- source.location_id:
 - 左侧: STATION_PIPELINE1_INPUT1 或 STATION_PIPELINE1_INPUT2
 - 右侧: STATION_PIPELINE2_INPUT1
- source.location_type: INPUT_PLATFORM （进料平台）
- target.location_id: STATION_NG_PLATFORM1 或 STATION_NG_PLATFORM2 （NG平台）
- target.location_type: NG_PLATFORM （NG平台）

响应示例:

```
{
  "status": "SUCCESS",
  "message": "Accepted",
  "code": "200",
  "trace_id": "DEV-LOG-1702627300000"
}
```

3.1.4 移料命令（从流水线进料位置流到流水线出料位置）

使用场景: 将物料从流水线进料位置传输到出料位置

请求示例:

```
{
  "device_id": "PIPELINE01",
  "command_id": "CMD-20251215-1001",
  "timestamp": "1702627300000",
  "task_type": "MOVE_FORWARD",
  "priority": 1,
  "timeout": 30000,
  "source": {
    "location_id": "STATION_PIPELINE_INPUT1",
    "location_type": "PIPELINE_PLATFORM"
  },
  "target": {
    "location_id": "STATION_PIPELINE_OUTPUT1",
    "location_type": "PIPELINE_PLATFORM"
  }
}
```

字段说明:

- device_id: PIPELINE01 (左侧流水线) 或 PIPELINE02 (右侧流水线)
- task_type: MOVE_FORWARD (前进任务)
- source.location_id: 流水线进料位置
- source.location_type: PIPELINE_PLATFORM (流水线平台)
- target.location_id: 流水线出料位置
- target.location_type: PIPELINE_PLATFORM (流水线平台)

响应示例:

```
{
  "status": "SUCCESS",
  "message": "Accepted",
  "code": "200",
  "trace_id": "DEV-LOG-1702627300000"
}
```

3.1.5 出料命令 (从流水线出料位置取出物料放到单层货架料箱的单元格)

使用场景: 将物料从流水线出料位置放置到料箱指定格子

请求示例:


```
{
  "device_id": "ARM02",
  "command_id": "CMD-20251215-1002",
  "timestamp": "1702627300000",
  "task_type": "PICK_AND_PUT",
  "priority": 1,
  "timeout": 30000,
  "source": {
    "location_id": "STATION_PIPELINE1_OUTPUT1",
    "location_type": "PIPELINE_PLATFORM"
  },
  "target": {
    "location_id": "STATION_OUTPUT1",
    "location_type": "BIN",
    "rack_id": "RACK_001",
    "bin_id": "BIN_104",
    "bin_type": "三格箱",
    "bin_cell_location": "1",
    "reel_layer": "15",
    "reel_thickness": "20",
    "reel_diameter": "15inch",
    "reel_totalthickness": "300"
  }
}
```

字段说明:

- device_id: ARM02 （左侧出料机械臂）或 ARM04 （右侧出料机械臂）
- task_type: PICK_AND_PUT （搬运任务）
- source.location_id:
 - 左侧: STATION_PIPELINE1_OUTPUT1
 - 右侧: STATION_PIPELINE2_OUTPUT1
- source.location_type: PIPELINE_PLATFORM （流水线平台）
- target.location_id: STATION_OUTPUT1 （出料位置）
- target.location_type: BIN （料箱）
- target.rack_id: RACK_001 （料架编号, ECS追溯用）
- target.bin_id: BIN_104 （箱体编号, ECS追溯用）
- target.bin_type: 三格箱 （料箱类型）
- target.bin_cell_location: 1 （料箱格子位置, 三格箱取值: 1,2,7; 六格箱取值: 1,2,3,4,5,6）
- target.reel_layer: 15 （目标料箱料盘层数）
- target.reel_thickness: 20 （当前料盘厚度, mm）
- target.reel_diameter: 15inch （当前料盘直径）
- target.reel_totalthickness: 300 （当前料箱料盘总厚度）

响应示例:

```
{
  "status": "SUCCESS",
  "message": "Accepted",
  "code": "200",
  "trace_id": "DEV-LOG-1702627300000"
}
```

3.2 取消命令

接口地址: POST /api/v1/command/cancel

功能说明: 取消已发送但未执行的命令

请求示例:

```
{
  "command_id": "CMD-20251215-1001"
}
```

字段说明:

- command_id: 要取消的命令ID

响应示例:

```
{
  "status": "SUCCESS",
  "message": "Cancelled",
  "code": "200",
  "trace_id": "DEV-LOG-CMD-20251215-1001"
}
```

注意事项:

- 只能取消状态为"等待执行"或"执行中"的命令
- 已完成或已取消的命令无法再次取消
- 取消命令本身会立即返回结果，实际命令取消状态需通过查询设备状态确认

4. 查询接口

4.1 查询设备状态

接口地址: GET /api/v1/status

功能说明: 查询所有流水线和机械臂的当前状态

请求参数: 无

响应示例:

```

{
  "status": [
    {
      "device_id": "PIPELINE01",
      "timestamp": 1702627300000,
      "current_cmd_id": "CMD-20251215-1001",
      "error_code": "NONE",
      "status": "RUNNING"
    },
    {
      "device_id": "ARM01",
      "timestamp": 1702627300000,
      "current_cmd_id": "CMD-20251215-1002",
      "error_code": "NONE",
      "status": "RUNNING"
    },
    {
      "device_id": "ARM02",
      "timestamp": 1702627300000,
      "current_cmd_id": "CMD-20251215-1003",
      "error_code": "NONE",
      "status": "RUNNING"
    },
    {
      "device_id": "PIPELINE02",
      "timestamp": 1702627300000,
      "current_cmd_id": "CMD-20251215-1004",
      "error_code": "NONE",
      "status": "RUNNING"
    },
    {
      "device_id": "ARM03",
      "timestamp": 1702627300000,
      "current_cmd_id": "CMD-20251215-1005",
      "error_code": "NONE",
      "status": "RUNNING"
    },
    {
      "device_id": "ARM04",
      "timestamp": 1702627300000,
      "current_cmd_id": "CMD-20251215-1006",
      "error_code": "NONE",
      "status": "RUNNING"
    }
  ]
}

```

字段说明:

- status : 设备状态数组, 包含6个设备的状态
 - 左侧半边: PIPELINE01 + ARM01 + ARM02
 - 右侧半边: PIPELINE02 + ARM03 + ARM04
- device_id : 设备ID
- timestamp : 状态更新时间戳 (毫秒)
- current_cmd_id : 当前正在执行的命令ID
- error_code : 错误码 (NONE 表示无错误)
- status : 设备状态 (IDLE / RUNNING / ERROR / OFFLINE)

设备状态说明:

- IDLE : 设备空闲，可以接收新命令
- RUNNING : 设备正在执行任务
- ERROR : 设备出现错误，需要处理
- OFFLINE : 设备离线，无法通信

5. 回调接口

5.1 任务结果回传 (Report Result)

接口地址: POST [WES回调地址]

功能说明: 设备完成命令后，向WES回传任务执行结果

5.1.1 进料NG结果回传 (扫码NG)

请求示例:

```
{
  "command_id": "CMD-20251215-1001",
  "device_id": "ARM01",
  "result": "SUCCESS",
  "finish_time": 1702627250000,
  "message": "任务执行成功",
  "data": {
    "actual_qty": 1,
    "location": "STATION_NG_PLATFORM1",
    "pick_and_put_result": "PUT_FINISHED"
  },
  "error_detail": {
    "error_code": "0",
    "error_message": ""
  }
}
```

字段说明:

- command_id : 原指令ID
- device_id : 设备ID
- result : 执行结果 (SUCCESS / FAILED)
- finish_time : 完成时间戳 (毫秒)
- message : 业务回传信息
- data.actual_qty : 实际搬运数量
- data.location : 实际搬运位置
- data.pick_and_put_result : 具体结果 (PUT_FINISHED 表示放料完成)
- error_detail.error_code : 错误码 (0表示无错误)
- error_detail.error_message : 具体错误信息

WES响应示例:

```
{
  "status": "SUCCESS",
  "message": "ACK",
  "code": "200",
  "trace_id": "DEV-LOG-1702627300000"
}
```

5.1.2 进料OK结果回传（尺寸检测和测厚OK）

请求示例:

```
{
  "command_id": "CMD-20251215-1001",
  "device_id": "ARM01",
  "result": "SUCCESS",
  "finish_time": 1702627250000,
  "message": "任务执行成功",
  "data": {
    "actual_qty": 1,
    "location": "STATION_PIPELINE1_INPUT1",
    "barcode1": "PKG1_12345678",
    "barcode2": "PKG2_12345678",
    "barcode3": "PKG3_12345678",
    "barcode4": "PKG4_12345678",
    "barcode5": "PKG5_12345678",
    "barcode6": "PKG6_12345678",
    "reel_diameter": "15inch",
    "reel_thickness": "20",
    "pick_and_put_result": "PUT_FINISHED"
  },
  "error_detail": {
    "error_code": "0",
    "error_message": ""
  }
}
```

字段说明:

- data.barcode1 ~ barcode6 : 扫码获取的6个条码信息
- data.reel_diameter : 料盘尺寸 (如: 15inch)
- data.reel_thickness : 料盘厚度 (mm)
- 其他字段同进料NG结果回传

WES响应示例:

```
{
  "status": "SUCCESS",
  "message": "ACK",
  "code": "200",
  "trace_id": "DEV-LOG-1702627300000"
}
```

5.1.3 进料NG结果回传（尺寸检测/测厚NG）

请求示例:

```
{
  "command_id": "CMD-20251215-1001",
  "device_id": "ARM01",
  "result": "FAILED",
  "finish_time": 1702627250000,
  "message": "任务执行失败",
  "data": {
    "actual_qty": 1,
    "location": "STATION_PIPELINE1_INPUT1",
    "barcode1": "PKG1_12345678",
    "barcode2": "PKG2_12345678",
    "barcode3": "PKG3_12345678",
    "barcode4": "PKG4_12345678",
    "barcode5": "PKG5_12345678",
    "barcode6": "PKG6_12345678",
    "reel_diameter": "15inch",
    "reel_thickness": "20",
    "pick_and_put_result": "PUT_FINISHED"
  },
  "error_detail": {
    "error_code": "1001",
    "error_message": "料盘尺寸检测异常"
  }
}
```

字段说明:

- result：FAILED（任务失败）
- error_detail.error_code：1001（料盘尺寸检测异常）或 1002（料盘厚度检测异常）
- error_detail.error_message：具体错误信息
- 其他字段同前

WES响应示例:

```
{
  "status": "SUCCESS",
  "message": "ACK",
  "code": "200",
  "trace_id": "DEV-LOG-1702627300000"
}
```

5.1.4 移料结果回传

请求示例:

```
{
  "command_id": "CMD-20251215-1001",
  "device_id": "ARM02",
  "result": "SUCCESS",
  "finish_time": 1702627250000,
  "data": {
    "actual_qty": 1,
    "pick_and_put_result": "MOVE_FINISHED"
  },
  "error_detail": {
    "error_code": "0",
    "error_message": ""
  }
}
```

字段说明:

- data.pick_and_put_result: MOVE_FINISHED 表示移料完成
- 其他字段同前

WES响应示例:

```
{
  "status": "SUCCESS",
  "message": "ACK",
  "code": "200",
  "trace_id": "DEV-LOG-1702627300000"
}
```

5.1.5 出料结果回传

请求示例:

```
{
  "command_id": "CMD-20251215-1001",
  "device_id": "ARM02",
  "result": "SUCCESS",
  "finish_time": 1702627250000,
  "data": {
    "actual_qty": 1,
    "pick_and_put_result": "PUT_FINISHED"
  },
  "error_detail": {
    "error_code": "0",
    "error_message": ""
  }
}
```

字段说明:

- data.pick_and_put_result: PUT_FINISHED 表示放料完成
- 其他字段同前

WES响应示例:

```
{
  "status": "SUCCESS",
  "message": "ACK",
  "code": "200",
  "trace_id": "DEV-LOG-1702627300000"
}
```

6. 事件推送接口

6.1 事件推送

接口地址: POST [WES事件上报地址]

功能说明: 设备向WES推送事件（如扫码完成、急停等）

6.1.1 扫码完成事件推送

请求示例:

```
{
  "device_id": "ARM01",
  "event_type": "SCAN_COMPLETED",
  "timestamp": 1702627300000,
  "data": {
    "location": "STATION_INPUT1",
    "barcode1": "PKG1_12345678",
    "barcode2": "PKG2_12345678",
    "barcode3": "PKG3_12345678",
    "barcode4": "PKG4_12345678",
    "barcode5": "PKG5_12345678",
    "barcode6": "PKG6_12345678"
  }
}
```

字段说明:

- device_id: 设备ID
- event_type: 事件类型（SCAN_COMPLETED 表示扫码完成，ESTOP_PRESSED 表示急停）
- timestamp: 事件发生时间戳（毫秒）
- data.location: 扫描位置（STATION_INPUT1 表示串杆扫码位置）
- data.barcode1 ~ barcode6: 扫码获取的6个条码信息

WES响应示例:

```
{
  "status": "SUCCESS",
  "message": "",
  "code": "200",
  "trace_id": "DEV-LOG-1702627300000"
}
```


6.1.2 急停事件推送

请求示例:

```
{
  "device_id": "ARM01",
  "event_type": "ESTOP_PRESSED",
  "timestamp": 1702627300000,
  "data": null
}
```

字段说明:

- event_type : ESTOP_PRESSED 表示急停按下
- data : 急停事件无业务数据

WES响应示例:

```
{
  "status": "SUCCESS",
  "message": "",
  "code": "200",
  "trace_id": "DEV-LOG-1702627300000"
}
```

7. 错误处理

7.1 HTTP状态码

状态码	说明	处理建议
200	请求成功	正常处理响应数据
400	参数错误	检查请求参数格式和必填项
503	设备忙/故障	等待后重试或联系设备维护人员

7.2 业务错误码

错误码	说明	处理建议
NONE	无错误	正常状态
0	无错误	任务成功执行
1001	料盘尺寸检测异常	检查料盘尺寸，将物料移到NG缓存位
1002	料盘厚度检测异常	检查料盘厚度，将物料移到NG缓存位
2001	扫码异常	检查条码质量和设备
2002	搬运失败	检查物料位置和机械臂状态
2003	料箱已满	更换料箱或选择其他位置
9999	未知错误	联系设备维护人员

7.3 错误处理示例

```
public async Task<ApiResult> SendCommandWithRetry(PickAndPutCommand command, int maxRetries = 3)
{
    int retryCount = 0;

    while (retryCount < maxRetries)
    {
        try
        {
            var response = await _httpClient.PostAsJsonAsync("/api/v1/command", command);
            var apiResult = await response.Content.ReadFromJsonAsync<ApiResult>();

            if (apiResult?.Code == "200")
            {
                return apiResult;
            }
            else if (apiResult?.Code == "400")
            {
                // 参数错误, 不重试
                throw new ArgumentException($"参数错误: {apiResult.Message}");
            }
            else if (apiResult?.Code == "503")
            {
                // 设备忙/故障, 等待后重试
                retryCount++;
                await Task.Delay(2000);
                continue;
            }
        }
        catch (Exception ex)
        {
            retryCount++;
            if (retryCount >= maxRetries)
            {
                throw;
            }
            await Task.Delay(1000);
        }
    }

    throw new InvalidOperationException($"命令发送失败, 已重试{maxRetries}次");
}
```

8. 完整示例代码

8.1 C#数据模型定义

参考 `Smt.Classifier.Models.cs` 文件中的完整定义。

8.2 SmtClassifierApiClient API客户端

```
using System;
using System.Net.Http;
using System.Text;
using System.Text.Json;
using System.Threading.Tasks;
using Smt.Classifier.Models;

namespace Smt.Classifier.Api
{
    /// <summary>
    /// SMT粗分机API客户端
    /// </summary>
    public class SmtClassifierApiClient : IDisposable
    {
        private readonly HttpClient _httpClient;
        private readonly string _baseUrl;
        private readonly JsonSerializerOptions _jsonOptions;

        public SmtClassifierApiClient(string baseUrl, TimeSpan? timeout = null)
        {
            _baseUrl = baseUrl.TrimEnd('/');
            _httpClient = new HttpClient();

            if (timeout.HasValue)
            {
                _httpClient.Timeout = timeout.Value;
            }

            _jsonOptions = new JsonSerializerOptions
            {
                PropertyNamingPolicy = JsonNamingPolicy.SnakeCaseLower,
                WriteIndented = true,
                DefaultIgnoreCondition = System.Text.Json.Serialization.JsonIgnoreCondition.WhenWritingNull
            };
        }

        /// <summary>
        /// 查询设备状态
        /// </summary>
        public async Task<ClassifierDeviceStatusResponse> GetDeviceStatusAsync()
        {
            var response = await _httpClient.GetAsync($"{_baseUrl}/api/v1/status");
            response.EnsureSuccessStatusCode();

            var content = await response.Content.ReadAsStringAsync();
            var result = JsonSerializer.Deserialize<ClassifierDeviceStatusResponse>(content, _jsonOptions);

            return result ?? new ClassifierDeviceStatusResponse();
        }

        /// <summary>
        /// 发送搬运命令
        /// </summary>
        public async Task<ApiResult> SendPickAndPutCommandAsync(PickAndPutCommand command)
        {
            var json = JsonSerializer.Serialize(command, _jsonOptions);
            var content = new StringContent(json, Encoding.UTF8, "application/json");
```

```

        var response = await _httpClient.PostAsync($"{_baseUrl}/api/v1/command", content);
        var responseContent = await response.Content.ReadAsStringAsync();

        var apiResult = JsonSerializer.Deserialize<ApiResult>(responseContent, _jsonOptions);
        return apiResult ?? new ApiResult();
    }

    /// <summary>
    /// 取消命令
    /// </summary>
    public async Task<ApiResult> CancelCommandAsync(string commandId)
    {
        var request = new CancelCommandRequest { CommandId = commandId };
        var json = JsonSerializer.Serialize(request, _jsonOptions);
        var content = new StringContent(json, Encoding.UTF8, "application/json");

        var response = await _httpClient.PostAsync($"{_baseUrl}/api/v1/command/cancel", content);
        var responseContent = await response.Content.ReadAsStringAsync();

        var apiResult = JsonSerializer.Deserialize<ApiResult>(responseContent, _jsonOptions);
        return apiResult ?? new ApiResult();
    }

    // 命令工厂方法

    /// <summary>
    /// 创建进料NG命令（扫码NG：从串杆放到NG缓存位）
    /// </summary>
    public static PickAndPutCommand CreateInputScanNgCommand(
        string deviceId = DeviceIdConstants.ARM01,
        string ngPlatform = DeviceIdConstants.STATION_NG_PLATFORM1,
        string? commandId = null,
        int priority = 1,
        int timeout = 30000)
    {
        return new PickAndPutCommand
        {
            DeviceId = deviceId,
            CommandId = commandId ?? $"CMD-{DateTime.Now:yyyyMMdd}-{Guid.NewGuid():N}",
            Timestamp = DateTimeOffset.UtcNow.ToUnixTimeMilliseconds().ToString(),
            TaskType = TaskTypeEnum.PICK_AND_PUT,
            Priority = priority,
            Timeout = timeout,
            Source = new PickAndPutLocationInfo
            {
                LocationId = DeviceIdConstants.STATION_INPUT1,
                LocationType = LocationTypeEnum.INPUT_PLATFORM
            },
            Target = new PickAndPutLocationInfo
            {
                LocationId = ngPlatform,
                LocationType = LocationTypeEnum.NG_PLATFORM
            }
        };
    }

    /// <summary>
    /// 创建进料OK命令（扫码OK：从串杆放到流水线进料位置）
    /// </summary>
    public static PickAndPutCommand CreateInputScanOkCommand(

```

```

        string deviceId = DeviceIdConstants.ARM01,
        string pipelineInput = DeviceIdConstants.STATION_PIPELINE1_INPUT1,
        string? commandId = null,
        int priority = 1,
        int timeout = 30000)
    {
        return new PickAndPutCommand
        {
            DeviceId = deviceId,
            CommandId = commandId ?? $"CMD-{DateTime.Now:yyyyMMdd}-{Guid.NewGuid():N}",
            Timestamp = DateTimeOffset.UtcNow.ToUnixTimeMilliseconds().ToString(),
            TaskType = TaskTypeEnum.PICK_AND_PUT,
            Priority = priority,
            Timeout = timeout,
            Source = new PickAndPutLocationInfo
            {
                LocationId = DeviceIdConstants.STATION_INPUT1,
                LocationType = LocationTypeEnum.INPUT_PLATFORM
            },
            Target = new PickAndPutLocationInfo
            {
                LocationId = pipelineInput,
                LocationType = LocationTypeEnum.PIPELINE_PLATFORM
            }
        };
    }
}

```

/// <summary>

/// 创建进料NG命令（尺寸检测/测厚NG：从流水线进料位置放到NG缓存位）

/// </summary>

```

public static PickAndPutCommand CreateInputSizeNgCommand(
    string deviceId = DeviceIdConstants.ARM01,
    string pipelineInput = DeviceIdConstants.STATION_PIPELINE1_INPUT1,
    string ngPlatform = DeviceIdConstants.STATION_NG_PLATFORM1,
    string? commandId = null,
    int priority = 1,
    int timeout = 30000)
{
    return new PickAndPutCommand
    {
        DeviceId = deviceId,
        CommandId = commandId ?? $"CMD-{DateTime.Now:yyyyMMdd}-{Guid.NewGuid():N}",
        Timestamp = DateTimeOffset.UtcNow.ToUnixTimeMilliseconds().ToString(),
        TaskType = TaskTypeEnum.PICK_AND_PUT,
        Priority = priority,
        Timeout = timeout,
        Source = new PickAndPutLocationInfo
        {
            LocationId = pipelineInput,
            LocationType = LocationTypeEnum.INPUT_PLATFORM
        },
        Target = new PickAndPutLocationInfo
        {
            LocationId = ngPlatform,
            LocationType = LocationTypeEnum.NG_PLATFORM
        }
    };
}

```

/// <summary>

```

/// 创建移料命令（从流水线进料位置流到流水线出料位置）
/// </summary>
public static PickAndPutCommand CreateMoveForwardCommand(
    string deviceId = DeviceIdConstants.PIPELINE01,
    string pipelineInput = DeviceIdConstants.STATION_PIPELINE_INPUT1,
    string pipelineOutput = DeviceIdConstants.STATION_PIPELINE_OUTPUT1,
    string? commandId = null,
    int priority = 1,
    int timeout = 30000)
{
    return new PickAndPutCommand
    {
        DeviceId = deviceId,
        CommandId = commandId ?? $"CMD-{DateTime.Now:yyyyMMdd}-{Guid.NewGuid():N}",
        Timestamp = DateTimeOffset.UtcNow.ToUnixTimeMilliseconds().ToString(),
        TaskType = TaskTypeEnum.MOVE_FORWARD,
        Priority = priority,
        Timeout = timeout,
        Source = new PickAndPutLocationInfo
        {
            LocationId = pipelineInput,
            LocationType = LocationTypeEnum.PIPELINE_PLATFORM
        },
        Target = new PickAndPutLocationInfo
        {
            LocationId = pipelineOutput,
            LocationType = LocationTypeEnum.PIPELINE_PLATFORM
        }
    };
}

/// <summary>
/// 创建出料命令（从流水线出料位置放到料箱）
/// </summary>
public static PickAndPutCommand CreateOutputCommand(
    string binCellLocation,
    int reellayer,
    string reelThickness,
    string reelDiameter,
    string reelTotalThickness,
    string deviceId = DeviceIdConstants.ARM02,
    string pipelineOutput = DeviceIdConstants.STATION_PIPELINE1_OUTPUT1,
    string? commandId = null,
    int priority = 1,
    int timeout = 30000)
{
    return new PickAndPutCommand
    {
        DeviceId = deviceId,
        CommandId = commandId ?? $"CMD-{DateTime.Now:yyyyMMdd}-{Guid.NewGuid():N}",
        Timestamp = DateTimeOffset.UtcNow.ToUnixTimeMilliseconds().ToString(),
        TaskType = TaskTypeEnum.PICK_AND_PUT,
        Priority = priority,
        Timeout = timeout,
        Source = new PickAndPutLocationInfo
        {
            LocationId = pipelineOutput,
            LocationType = LocationTypeEnum.PIPELINE_PLATFORM
        },
        Target = new PickAndPutLocationInfo
    }
}

```

```
{
    LocationId = DeviceIdConstants.STATION_OUTPUT1,
    LocationType = LocationTypeEnum.BIN,
    RackId = "RACK_001",
    BinId = "BIN_104",
    BinType = BinTypeEnum.THREE_CELL_BIN,
    BinCellLocation = binCellLocation,
    ReelLayer = reelLayer.ToString(),
    ReelThickness = reelThickness,
    ReelDiameter = reelDiameter,
    ReelTotalThickness = reelTotalThickness
}
};
}

public void Dispose()
{
    _httpClient?.Dispose();
}
}
}
```

8.3 使用示例

```
using System;
using System.Threading.Tasks;
using Smt.Classifier.Api;
using Smt.Classifier.Models;

class Program
{
    static async Task Main(string[] args)
    {
        // 创建API客户端（替换为实际的设备IP地址）
        using var client = new SmtClassifierApiClient("http://192.168.1.100:8080");

        try
        {
            // 示例1: 查询设备状态
            Console.WriteLine("=== 查询设备状态 ===");
            var status = await client.GetDeviceStatusAsync();
            foreach (var device in status.Status)
            {
                Console.WriteLine($"设备ID: {device.DeviceId}, 状态: {device.Status}, 当前命令: {device.CurrentCmdId}");
            }
            Console.WriteLine();

            // 示例2: 进料OK流程（左侧）
            Console.WriteLine("=== 进料OK流程（左侧） ===");
            var inputOkCommand = SmtClassifierApiClient.CreateInputScanOkCommand(
                deviceId: DeviceIdConstants.ARM01,
                pipelineInput: DeviceIdConstants.STATION_PIPELINE1_INPUT1
            );
            var result1 = await client.SendPickAndPutCommandAsync(inputOkCommand);
            Console.WriteLine($"进料OK命令发送结果: {result1.Status} - {result1.Message}");
            Console.WriteLine();

            // 示例3: 进料NG流程（尺寸检测NG, 左侧）
            Console.WriteLine("=== 进料NG流程（尺寸检测NG, 左侧） ===");
            var inputSizeNgCommand = SmtClassifierApiClient.CreateInputSizeNgCommand(
                deviceId: DeviceIdConstants.ARM01,
                pipelineInput: DeviceIdConstants.STATION_PIPELINE1_INPUT1,
                ngPlatform: DeviceIdConstants.STATION_NG_PLATFORM1
            );
            var result2 = await client.SendPickAndPutCommandAsync(inputSizeNgCommand);
            Console.WriteLine($"进料NG命令发送结果: {result2.Status} - {result2.Message}");
            Console.WriteLine();

            // 示例4: 移料流程（左侧）
            Console.WriteLine("=== 移料流程（左侧） ===");
            var moveCommand = SmtClassifierApiClient.CreateMoveForwardCommand(
                deviceId: DeviceIdConstants.PIPELINE01
            );
            var result3 = await client.SendPickAndPutCommandAsync(moveCommand);
            Console.WriteLine($"移料命令发送结果: {result3.Status} - {result3.Message}");
            Console.WriteLine();

            // 示例5: 出料流程（左侧）
            Console.WriteLine("=== 出料流程（左侧） ===");
            var outputCommand = SmtClassifierApiClient.CreateOutputCommand(
                binCellLocation: "1",
                reelLayer: 15,
            );
        }
    }
}
```



```

        reelThickness: "20",
        reelDiameter: "15inch",
        reelTotalThickness: "300",
        deviceId: DeviceIdConstants.ARM02,
        pipelineOutput: DeviceIdConstants.STATION_PIPELINE1_OUTPUT1
    );
    var result4 = await client.SendPickAndPutCommandAsync(outputCommand);
    Console.WriteLine($"出料命令发送结果: {result4.Status} - {result4.Message}");
    Console.WriteLine();

    // 示例6: 取消命令
    Console.WriteLine("=== 取消命令 ===");
    var cancelResult = await client.CancelCommandAsync(inputOkCommand.CommandId);
    Console.WriteLine($"取消命令结果: {cancelResult.Status} - {cancelResult.Message}");
    Console.WriteLine();

    // 示例7: 进料OK流程（右侧）
    Console.WriteLine("=== 进料OK流程（右侧） ===");
    var inputOkCommandRight = SmtClassifierApiClient.CreateInputScanOkCommand(
        deviceId: DeviceIdConstants.ARM03,
        pipelineInput: DeviceIdConstants.STATION_PIPELINE2_INPUT1
    );
    var result5 = await client.SendPickAndPutCommandAsync(inputOkCommandRight);
    Console.WriteLine($"进料OK命令发送结果（右侧）: {result5.Status} - {result5.Message}");
    Console.WriteLine();

    // 示例8: 出料流程（右侧）
    Console.WriteLine("=== 出料流程（右侧） ===");
    var outputCommandRight = SmtClassifierApiClient.CreateOutputCommand(
        binCellLocation: "1",
        reelLayer: 15,
        reelThickness: "20",
        reelDiameter: "15inch",
        reelTotalThickness: "300",
        deviceId: DeviceIdConstants.ARM04,
        pipelineOutput: DeviceIdConstants.STATION_PIPELINE2_OUTPUT1
    );
    var result6 = await client.SendPickAndPutCommandAsync(outputCommandRight);
    Console.WriteLine($"出料命令发送结果（右侧）: {result6.Status} - {result6.Message}");
    Console.WriteLine();
}
catch (Exception ex)
{
    Console.WriteLine($"发生错误: {ex.Message}");
}
}
}

```

9. 附录

9.1 设备ID清单

设备ID	设备类型	所属半边	说明
PIPELINE01	流水线	左侧	负责物料传输（单串杆和四串杆）

设备ID	设备类型	所属半边	说明
ARM01	机械臂	左侧	进料机械臂（扫码、尺寸检测、测厚）
ARM02	机械臂	左侧	出料机械臂
PIPELINE02	流水线	右侧	负责物料传输（单串杆）
ARM03	机械臂	右侧	进料机械臂（扫码、尺寸检测、测厚）
ARM04	机械臂	右侧	出料机械臂

9.2 位置ID清单

位置ID	位置类型	所属半边	说明
STATION_INPUT1	INPUT_PLATFORM	通用	串杆扫码位置
STATION_PIPELINE1_INPUT1	PIPELINE_PLATFORM	左侧	流水线1进料位置1
STATION_PIPELINE1_INPUT2	PIPELINE_PLATFORM	左侧	流水线1进料位置2（四串杆）
STATION_PIPELINE1_OUTPUT1	PIPELINE_PLATFORM	左侧	流水线1出料位置
STATION_PIPELINE2_INPUT1	PIPELINE_PLATFORM	右侧	流水线2进料位置
STATION_PIPELINE2_OUTPUT1	PIPELINE_PLATFORM	右侧	流水线2出料位置
STATION_NG_PLATFORM1	NG_PLATFORM	通用	NG平台1
STATION_NG_PLATFORM2	NG_PLATFORM	通用	NG平台2
STATION_OUTPUT1	BIN	通用	出料位置（料箱）

9.3 料箱类型说明

料箱类型	格子位置取值范围	说明
三格箱	1, 2, 7	有3个格子
六格箱	1, 2, 3, 4, 5, 6	有6个格子

9.4 料盘直径规格

规格	说明
7inch	7英寸料盘
13inch	13英寸料盘
15inch	15英寸料盘（常用）

9.5 任务类型枚举

枚举值	说明
PICK_AND_PUT	搬运任务（从源位置抓取物料放到目标位置）
SCAN	扫码任务
TEST	测试任务

枚举值	说明
MOVE_FORWARD	前进任务（流水线向前移动）
MOVE_BACKWARD	后退任务（流水线向后移动）
MOVE_LEFT	左移任务
MOVE_RIGHT	右移任务

9.6 位置类型枚举

枚举值	说明	必填字段
INPUT_PLATFORM	进料平台	location_id, location_type
NG_PLATFORM	NG平台	location_id, location_type
PIPELINE_PLATFORM	流水线平台	location_id, location_type
OUTPUT_PLATFORM	出料平台	location_id, location_type
BIN	料箱	location_id, location_type, bin_type, bin_cell_location, reel_layer, reel_thickness, reel_diameter, reel_totalthickness

9.7 设备状态枚举

枚举值	说明	操作建议
IDLE	空闲	可以发送新命令
RUNNING	运行中	正在执行任务，请等待
ERROR	错误	需要处理错误后才能继续
OFFLINE	离线	检查设备连接

9.8 事件类型枚举

枚举值	说明	处理建议
SCAN_COMPLETED	扫码完成	根据扫码结果决定进料OK或NG
ESTOP_PRESSED	急停按下	立即停止所有任务，检查设备状态

9.9 业务错误码枚举

错误码	说明	处理建议
NONE	无错误	正常状态
0	无错误	任务成功执行
1001	料盘尺寸检测异常	检查料盘尺寸，将物料移到NG缓存位
1002	料盘厚度检测异常	检查料盘厚度，将物料移到NG缓存位
2001	扫码异常	检查条码质量和设备
2002	搬运失败	检查物料位置和机械臂状态
2003	料箱已满	更换料箱或选择其他位置

错误码	说明	处理建议
9999	未知错误	联系设备维护人员

9.10 术语表（中英文对照）

中文	英文	说明
粗分机	Classifier	SMT生产线中的自动化分拣设备
流水线	Pipeline	负责物料传输的传送带系统
机械臂	Robotic Arm	自动化搬运设备
串杆	Rod	用于放置待扫码物料的装置
单串杆	Single Rod	单个串杆配置
四串杆	Four Rods	四个串杆配置
进料	Input	物料进入系统的过程
出料	Output	物料离开系统的过程
料箱	Bin	用于存储物料的箱子
料盘	Reel	存放电子物料的盘状载体
条码	Barcode	物料的标识码
NG	No Good	不合格品
OK	Okay	合格品
尺寸检测	Size Detection	检测料盘尺寸是否合格
测厚	Thickness Measurement	测量料盘厚度
急停	Emergency Stop	紧急停止按钮
时间戳	Timestamp	事件发生的时间（毫秒）
命令ID	Command ID	命令的唯一标识
设备ID	Device ID	设备的唯一标识
左侧半边	Left Side	PIPELINE01/ARM01/ARM02（单串杆和四串杆）
右侧半边	Right Side	PIPELINE02/ARM03/ARM04（单串杆）

9.11 完整业务流程

9.11.1 进料OK流程 (尺寸检测OK)

1. 扫码事件 (设备 → WES)
 - 设备扫码获取条码信息
 - WES判断条码OK
2. 进料OK命令 (WES → 设备)
 - 从串杆位置 → 流水线进料位置
3. 任务结果回传 (设备 → WES)
 - 包含条码信息、料盘尺寸、料盘厚度
 - 尺寸检测和测厚OK
4. 移料命令 (WES → 设备)
 - 流水线进料位置 → 流水线出料位置
5. 移料结果回传 (设备 → WES)
6. 出料命令 (WES → 设备)
 - 流水线出料位置 → 料箱
7. 出料结果回传 (设备 → WES)

9.11.2 进料NG流程 (扫码NG)

1. 扫码事件 (设备 → WES)
 - 设备扫码获取条码信息
 - WES判断条码NG
2. 进料NG命令 (WES → 设备)
 - 从串杆位置 → NG缓存位
3. 进料NG结果回传 (设备 → WES)

9.11.3 进料NG流程 (尺寸检测/测厚NG)

1. 扫码事件 (设备 → WES)
 - 设备扫码获取条码信息
 - WES判断条码OK
2. 进料OK命令 (WES → 设备)
 - 从串杆位置 → 流水线进料位置
3. 任务结果回传 (设备 → WES)
 - 包含条码信息、料盘尺寸、料盘厚度
 - 尺寸检测或测厚NG
 - 错误码: 1001 (尺寸检测异常) 或 1002 (厚度检测异常)
4. 进料NG命令 (WES → 设备)
 - 从流水线进料位置 → NG缓存位
5. 进料NG结果回传 (设备 → WES)

9.12 完整JSON示例

进料OK流程完整示例（尺寸检测OK）

1. 发送扫码完成事件（设备 → WES）：

```
{
  "device_id": "ARM01",
  "event_type": "SCAN_COMPLETED",
  "timestamp": 1702627300000,
  "data": {
    "location": "STATION_INPUT1",
    "barcode1": "PKG1_12345678",
    "barcode2": "PKG2_12345678",
    "barcode3": "PKG3_12345678",
    "barcode4": "PKG4_12345678",
    "barcode5": "PKG5_12345678",
    "barcode6": "PKG6_12345678"
  }
}
```

2. WES响应扫码事件：

```
{
  "status": "SUCCESS",
  "message": "",
  "code": "200",
  "trace_id": "DEV-LOG-1702627300000"
}
```

3. WES发送进料OK命令（WES → 设备）：

```
{
  "device_id": "ARM01",
  "command_id": "CMD-20251215-1001",
  "timestamp": "1702627300000",
  "task_type": "PICK_AND_PUT",
  "priority": 1,
  "timeout": 30000,
  "source": {
    "location_id": "STATION_INPUT1",
    "location_type": "INPUT_PLATFORM"
  },
  "target": {
    "location_id": "STATION_PIPELINE1_INPUT1",
    "location_type": "PIPELINE_PLATFORM"
  }
}
```

4. 设备响应命令：

```
{
  "status": "SUCCESS",
  "message": "Accepted",
  "code": "200",
  "trace_id": "DEV-LOG-1702627300000"
}
```

5. 设备回传任务结果（尺寸检测OK）（设备 → WES）：

```
{
  "command_id": "CMD-20251215-1001",
  "device_id": "ARM01",
  "result": "SUCCESS",
  "finish_time": 1702627250000,
  "message": "任务执行成功",
  "data": {
    "actual_qty": 1,
    "location": "STATION_PIPELINE1_INPUT1",
    "barcode1": "PKG1_12345678",
    "barcode2": "PKG2_12345678",
    "barcode3": "PKG3_12345678",
    "barcode4": "PKG4_12345678",
    "barcode5": "PKG5_12345678",
    "barcode6": "PKG6_12345678",
    "reel_diameter": "15inch",
    "reel_thickness": "20",
    "pick_and_put_result": "PUT_FINISHED"
  },
  "error_detail": {
    "error_code": "0",
    "error_message": ""
  }
}
```

6. WES响应任务结果:

```
{
  "status": "SUCCESS",
  "message": "ACK",
  "code": "200",
  "trace_id": "DEV-LOG-1702627300000"
}
```

版本历史

版本	日期	说明
v2.0	2026-03-21	更新版本，增加尺寸检测和测厚功能，左右侧半边设备配置说明
v1.0	2026-03-21	初始版本，基础接口说明

文档结束